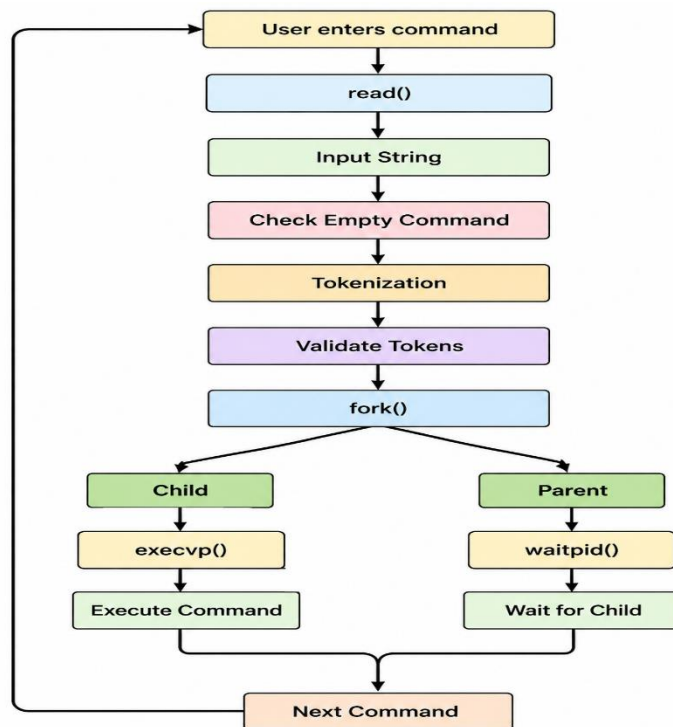


Refer the following link to understand the Parsing and Lexical analysis
<https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/>

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

Flow Chart:



```
alekhya@Alekhyareddy: ~/OS  X  +  v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano main_loop.c
alekhya@Alekhyareddy:~/OSSP$ gcc main_loop.c -o main_loop
alekhya@Alekhyareddy:~/OSSP$ ./main_loop
myshell> hello world
You entered: hello world
myshell> exit
Exiting shell...
alekhya@Alekhyareddy:~/OSSP$
```

```
alekhya@AlekhyaReddy: ~/OS × + v
GNU nano 7.2
#include <stdio.h>
#include <string.h>

int main()
{
    char command[100];

    while (1)
    {
        printf("myshell> ");

        fgets(command, sizeof(command), stdin);

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

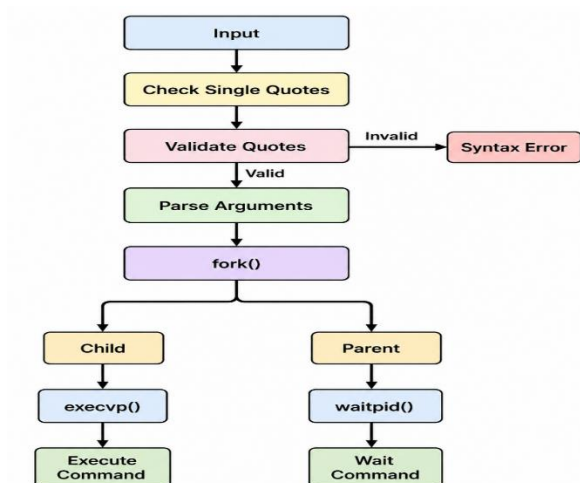
        printf("You entered: %s\n", command);
    }

    return 0;
}
```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Flow Chart:



Example Output:

Case1:

myShell> echo Hello

Parsing Result:

Argument 1: echo

Argument 2: Hello

Child PID: 3210

Hello

Command execution completed.

```
alekhya@Alekhyareddy:~/OSSP$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano task2_quotes.c
alekhya@Alekhyareddy:~/OSSP$ gcc task2_quotes.c -o task2_quotes
alekhya@Alekhyareddy:~/OSSP$ ./task2_quotes
myShell> echo Hello

Parsing Result:
Argument 1: echo
Argument 2: Hello

Child PID: 595
Hello

Command execution completed.
```

Case2:

myShell> echo 'Hello World

Syntax Error: Unmatched single quote.

```
myShell> echo 'Hello World'

Parsing Result:
Argument 1: echo
Argument 2: Hello World

Child PID: 598
Hello World

Command execution completed.

myShell> echo 'Hello World

Syntax Error: Unmatched single quote.
myShell> exit
Exiting shell...
alekhya@Alekhyareddy:~/OSSP$ |
```

```
alekhya@Alekhyareddy: ~/OS  ×  +  v
GNU nano 7.2
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

#define MAX_INPUT 256
#define MAX_ARGS 20

int parse_command(char *input, char *args[])
{
    int argc = 0;
    int in_single_quote = 0;
    char *p = input;

    while (*p != '\0')
    {
        /* Skip spaces outside quotes */
        while (*p == ' ' && !in_single_quote)
            p++;

        if (*p == '\0')
            break;

        char *start = p;

        if (*p == '\'')
        {
            /* Start single quote */
            in_single_quote = 1;
            start = ++p;

            while (*p != '\0' && *p != '\0')
                p++;

            if (*p == '\0')
            {
                printf("\nSyntax Error: Unmatched single quote.\n");
                return -1;
            }

            *p = '\0';
            args[argc++] = start;
            p++;
        }
        else
        {

```

```
alekhya@Alekhyareddy: ~/OS  ×  +  v
GNU nano 7.2
        else
        {
            start = p;

            while (*p != '\0' && *p != ' ')
                p++;

            if (*p != '\0')
            {
                *p = '\0';
                p++;
            }

            args[argc++] = start;
        }

        if (argc >= MAX_ARGS - 1)
            break;

        args[argc] = NULL;
        return argc;
    }

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1)
    {
        printf("myShell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        int argc = parse_command(input, args);

        if (argc == -1)
            continue;
    }
}
```

```
alekhya@Alekhyareddy: ~/OS × + v
GNU nano 7.2
        break;

    int argc = parse_command(input, args);

    if (argc == -1)
        continue;

    if (argc == 0)
        continue;

    printf("\nParsing Result:\n");

    for (int i = 0; i < argc; i++)
    {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        continue;
    }

    if (pid == 0)
    {
        printf("\nChild PID: %d\n", getpid());

        execvp(args[0], args);

        perror("Command execution failed");
        exit(1);
    }
    else
    {
        wait(NULL);

        printf("\nCommand execution completed.\n\n");
    }

    printf("Exiting shell...\n");

    return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut
^X Exit      ^R Read File  ^\ Replace    ^U Paste

2
Search
```

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Example Output:

Case1:

myShell> echo "Hello \$USER"

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello raghu

Child Process

PID = 3022

Hello raghu

Child process completed.

Case2:

myShell> echo "Hello World

Syntax Error: Unmatched double quote.

```
alekhya@AlekhyaReddy:~/OSSP$ cd ~/OSSP
alekhya@AlekhyaReddy:~/OSSP$ nano task3_double_quotes.c
alekhya@AlekhyaReddy:~/OSSP$ gcc task3_double_quotes.c -o task3_double_quotes
alekhya@AlekhyaReddy:~/OSSP$ ./task3_double_quotes
myShell> echo "Hello $USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello alekhya

Child Process
PID = 678

Hello alekhya

Child process completed.

myShell> echo $USER

--- Parsed Arguments ---
Argument 1: echo
Argument 2: $USER

Child Process
PID = 679

$USER

Child process completed.
```

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 256
#define MAX_ARGS 20

int parse_command(char *input, char *args[])
{
    int argc = 0;
    char *p = input;

    while (*p != '\0')
    {
        /* Skip spaces */
        while (*p == ' ')
            p++;

        if (*p == '\0')
            break;

        /* Double quoted argument */
        if (*p == '"')
        {
            char *start;

            p++;
            start = p;

            while (*p != '\0' && *p != '"')
                p++;

            if (*p == '\0')
            {
                printf("\nSyntax Error: Unmatched double quote.\n");
                return -1;
            }

            *p = '\0';

            /* Expand environment variables */
            char expanded[MAX_INPUT];
            int i = 0;
            int j = 0;

            while (start[i] != '\0' && j < MAX_INPUT - 1)
            {
                if (start[i] == '$')
                {
                    char variable[100];
                    int k = 0;

                    i++;

                    while (start[i] != '\0' &&
                        (start[i] == '_' ||
                         (start[i] >= 'A' && start[i] <= 'Z') ||
                         (start[i] >= 'a' && start[i] <= 'z')) ||

```

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 256
#define MAX_ARGS 20

int parse_command(char *input, char *args[])
{
    int argc = 0;
    char *p = input;

    while (*p != '\0')
    {
        /* Skip spaces */
        while (*p == ' ')
            p++;

        if (*p == '\0')
            break;

        /* Double quoted argument */
        if (*p == '"')
        {
            char *start;

            p++;
            start = p;

            while (*p != '\0' && *p != '"')
                p++;

            if (*p == '\0')
            {
                printf("\nSyntax Error: Unmatched double quote.\n");
                return -1;
            }

            *p = '\0';

            /* Expand environment variables */
            char expanded[MAX_INPUT];
            int i = 0;
            int j = 0;

            while (start[i] != '\0' && j < MAX_INPUT - 1)
            {
                if (start[i] == '$')
                {
                    char variable[100];
                    int k = 0;

                    i++;

                    while (start[i] != '\0' &&
                           (start[i] == '.' ||
                            (start[i] >= 'A' && start[i] <= 'Z') ||
                            (start[i] >= 'a' && start[i] <= 'z')))
                    {
                        variable[k++] = start[i++];
                    }

                    variable[k] = '\0';
                    char *value = getenv(variable);

                    if (value != NULL)
                    {
                        for (int x = 0; value[x] != '\0' &&
                             j < MAX_INPUT - 1; x++)
                        {
                            expanded[j++] = value[x];
                        }
                    }
                    else
                    {
                        expanded[j++] = start[i++];
                    }
                }
                else
                {
                    expanded[j++] = start[i++];
                }
            }

            expanded[j] = '\0';
            args[argc] = malloc(strlen(expanded) + 1);
            if (args[argc] == NULL)
                return -1;

            strcpy(args[argc], expanded);
            argc++;
            p++;
        }
        else
        {
            /* Normal unquoted argument */
            char *start = p;

            while (*p != '\0' && *p != ' ')
                p++;

            if (*p != '\0')
            {
                *p = '\0';

                args[argc] = malloc(strlen(start) + 1);
                if (args[argc] == NULL)
                    return -1;
            }
        }
    }
}
```

```
GNU nano 7.2

        while (start[i] != '\0' &&
               (start[i] == '.' ||
                (start[i] >= 'A' && start[i] <= 'Z') ||
                (start[i] >= 'a' && start[i] <= 'z')))
        {
            variable[k++] = start[i++];
        }

        variable[k] = '\0';
        char *value = getenv(variable);
        if (value != NULL)
        {
            for (int x = 0; value[x] != '\0' &&
                 j < MAX_INPUT - 1; x++)
            {
                expanded[j++] = value[x];
            }
        }
        else
        {
            expanded[j++] = start[i++];
        }
    }

    expanded[j] = '\0';
    args[argc] = malloc(strlen(expanded) + 1);
    if (args[argc] == NULL)
        return -1;

    strcpy(args[argc], expanded);
    argc++;
    p++;
}
else
{
    /* Normal unquoted argument */
    char *start = p;

    while (*p != '\0' && *p != ' ')
        p++;

    if (*p != '\0')
    {
        *p = '\0';

        args[argc] = malloc(strlen(start) + 1);
        if (args[argc] == NULL)
            return -1;
    }
}
```



```
alekhya@Alekhyareddy: ~/OS
GNU nano 7.2
    }
    args[argc] = malloc(strlen(start) + 1);
    if (args[argc] == NULL)
        return -1;
    strcpy(args[argc], start);
    argc++;
}
if (argc >= MAX_ARGS - 1)
    break;
}
args[argc] = NULL;
return argc;
}
int main()
{
    char input[MAX_INPUT];
    while (1)
    {
        char *args[MAX_ARGS];
        printf("myShell> ");
        fflush(stdout);
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strlen(input)] = '\0';
        if (strcmp(input, "exit") == 0)
            break;
        int argc = parse_command(input, args);
        if (argc == -1)
            continue;
        if (argc == 0)
            continue;
        printf("\n--- Parsed Arguments ---\n");
        for (int i = 0; i < argc; i++)
        {
            printf("Argument %d: %s\n", i + 1, args[i]);
        }
        pid_t pid = fork();
        if (pid < 0)
        {
            perror("fork");
        }
    }
}
^G Help      ^O Write Out  ^W Where Is   ^K Ctrl-C
^X Exit      ^R Read File  ^_ Replace    ^U Paste
```

```
        perror("fork");
    }
    else if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("PID = %d\n\n", getpid());
        execvp(args[0], args);
        perror("Command execution failed");
        exit(1);
    }
    else
    {
        wait(NULL);
        printf("\nChild process completed.\n");
    }
    for (int i = 0; i < argc; i++)
    {
        free(args[i]);
    }
    printf("\n");
}
printf("Exiting shell...\n");
return 0;
}
```

```

alekhya@Alekhyareddy:~/OSSP$ nano task3_double_quotes.c
alekhya@Alekhyareddy:~/OSSP$ ./task3_double_quotes
myShell> echo "Hello World"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Child Process
PID = 695

Hello World

Child process completed.

myShell> echo "Hello World

Syntax Error: Unmatched double quote.
myShell> |

```

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 256
#define MAX_ARGS 20
#define MAX_ARG_LEN 100

int parse_command(char *input, char *args[])
{
    int argc = 0;
    int i = 0;

    while (input[i] != '\0' && argc < MAX_ARGS - 1)
    {
        /* Skip spaces */
        while (input[i] == ' ')
            i++;

        if (input[i] == '\0')
            break;

        /* Allocate memory for argument */
        args[argc] = malloc(MAX_ARG_LEN);

        if (args[argc] == NULL)
        {
            printf("Memory allocation failed.\n");
            return -1;
        }

        int j = 0;

        /* Read one argument */
        while (input[i] != '\0')
        {
            /* Normal space ends the argument */
            if (input[i] == ' ')
                break;

            /* Backslash escapes the next character */
            if (input[i] == '\\')
            {
                i++;

                if (input[i] != '\0')
                {

```

^{^G} Help ^{^O} Write Out ^{^W} Where Is ^{^K} Cut
^{^X} Exit ^{^R} Read File ^{^_} Replace ^{^U} Paste

```
alekhya@Alekhyareddy: ~/OS × + v
GNU nano 7.2

        if (input[i] != '\0')
        {
            args[argc][j++] = input[i];
            i++;
        }
    }
    else
    {
        args[argc][j++] = input[i];
        i++;
    }

    if (j >= MAX_ARG_LEN - 1)
        break;
}

args[argc][j] = '\0';
argc++;

/* Skip separator spaces */
while (input[i] == ' ')
    i++;
}

args[argc] = NULL;
return argc;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;
    }

    ^G Help      ^O Write Out  ^W Where Is   ^K Cu
    ^X Exit      ^R Read File  ^\ Replace    ^U Pa
```

```
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        int argc = parse_command(input, args);

        if (argc < 0)
            continue;

        printf("\n--- Parsed Arguments ---\n");

        for (int i = 0; i < argc; i++)
        {
            printf("Argument %d: %s\n", i + 1, args[i]);
        }

        printf("\n");

        /* Free allocated memory */
        for (int i = 0; i < argc; i++)
        {
            free(args[i]);
        }

        printf("Exiting shell...\n");

        return 0;
    }

    ^G Help      ^O Write Out  ^W Where Is   ^K Cu
    ^X Exit      ^R Read File  ^\ Replace    ^U Pa
```

```
alekhya@AlekhyaReddy: ~/OS x + v
alekhya@AlekhyaReddy:~$ cd ~/OSSP
alekhya@AlekhyaReddy:~/OSSP$ nano task4_escape.c
alekhya@AlekhyaReddy:~/OSSP$ gcc task4_escape.c -o task4_escape
alekhya@AlekhyaReddy:~/OSSP$ ./task4_escape
myshell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello

myshell> echo Hello\ World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

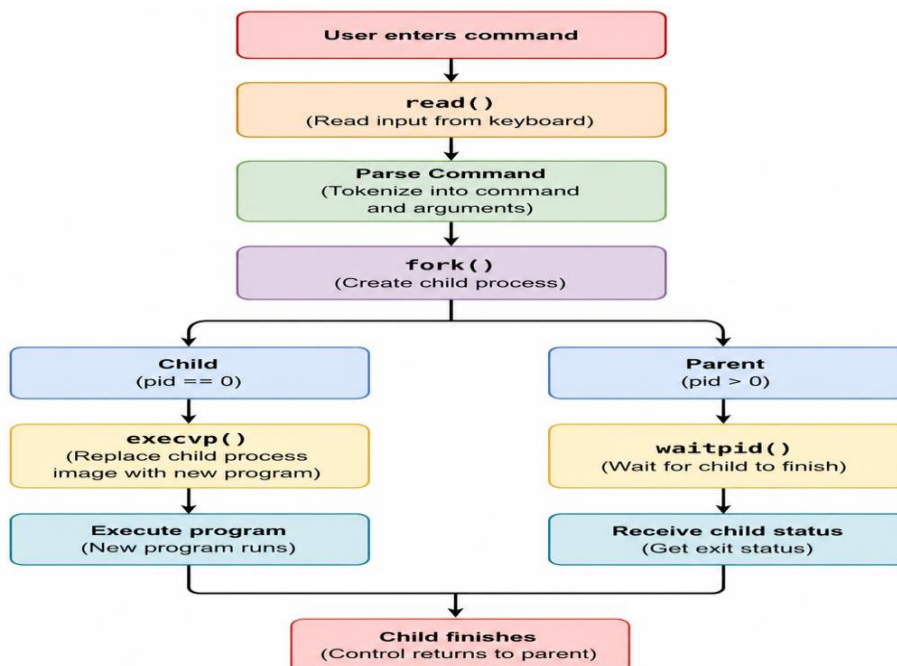
myshell> echo Hello\@World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello@World

myshell> |
```

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

Flow Chart:



Sample Output:

Case 1:

myshell\$ ls

Parent Process PID: 1200

Created Child PID: 1201

Child Process PID: 1201

file1.txt

file2.txt

Child exited with status: 0

Case 2:

myshell\$ ls @l

Parent Process PID: 82

Child Process PID: 83

Created Child PID: 83

ls: cannot access '@l': No such file or directory

Child exited with status: 2

```
alekhya@AlekhyaReddy: ~/OS x + v
alekhya@AlekhyaReddy:~$ cd ~/OSSP
alekhya@AlekhyaReddy:~/OSSP$ nano task5_process.c
alekhya@AlekhyaReddy:~/OSSP$ gcc -Wall -Wextra task5_process.c -o task5_process
alekhya@AlekhyaReddy:~/OSSP$ ./task5_process
myshell$ ls
Parent Process PID: 616
Created Child PID: 617
Child Process PID: 617
LICENSE bin main_loop.c task2_quotes task3_input task4_history task5_process
Makefile docs obj task2_quotes.c task3_input.c task4_history.c task5_process.c
README.md include scripts task3_double_quotes task4_escape task5_memory tests
assets main_loop src task3_double_quotes.c task4_escape.c task5_memory.c
Child exited with status: 0

myshell$ ls @l
Parent Process PID: 616
Created Child PID: 622
Child Process PID: 622
ls: cannot access '@l': No such file or directory
Child exited with status: 2

myshell$ pwd
Parent Process PID: 616
Created Child PID: 623
Child Process PID: 623
/home/alekhya/OSSP
Child exited with status: 0

myshell$ whoami
Parent Process PID: 616
Created Child PID: 624
Child Process PID: 624
alekhya
Child exited with status: 0

myshell$ ls -l
Parent Process PID: 616
Created Child PID: 626
Child Process PID: 626
total 220
-rw-r--r-- 1 alekhya alekhya 0 Aug 3 04:15 LICENSE
-rw-r--r-- 1 alekhya alekhya 0 Aug 3 04:15 Makefile
-rw-r--r-- 1 alekhya alekhya 0 Aug 3 04:15 README.md
```

alekhya@Alekhyareddy: ~/OS

Parent Process PID: 616

Created Child PID: 626

Child Process PID: 626

total 220

```
-rw-r--r-- 1 alekhya alekhya 0 Aug 3 04:15 LICENSE
-rw-r--r-- 1 alekhya alekhya 0 Aug 3 04:15 Makefile
-rw-r--r-- 1 alekhya alekhya 0 Aug 3 04:15 README.md
drwxr-xr-x 2 alekhya alekhya 4096 Aug 1 09:08 assets
drwxr-xr-x 2 alekhya alekhya 4096 Aug 4 05:32 bin
drwxr-xr-x 2 alekhya alekhya 4096 Aug 1 09:08 docs
drwxr-xr-x 2 alekhya alekhya 4096 Aug 1 09:07 include
-rwxr-xr-x 1 alekhya alekhya 16232 Aug 13 11:45 main_loop
-rw-r--r-- 1 alekhya alekhya 420 Aug 13 11:45 main_loop.c
drwxr-xr-x 2 alekhya alekhya 4096 Aug 1 09:07 obj
drwxr-xr-x 2 alekhya alekhya 4096 Aug 1 09:08 scripts
drwxr-xr-x 2 alekhya alekhya 4096 Aug 4 05:32 src
-rwxr-xr-x 1 alekhya alekhya 16608 Aug 13 11:56 task2_quotes
-rw-r--r-- 1 alekhya alekhya 2380 Aug 13 11:56 task2_quotes.c
-rwxr-xr-x 1 alekhya alekhya 16872 Aug 13 12:48 task3_double_quotes
-rw-r--r-- 1 alekhya alekhya 4088 Aug 13 12:53 task3_double_quotes.c
-rwxr-xr-x 1 alekhya alekhya 16488 Aug 7 08:09 task3_input
-rw-r--r-- 1 alekhya alekhya 1661 Aug 7 08:09 task3_input.c
-rwxr-xr-x 1 alekhya alekhya 16488 Aug 17 04:59 task4_escape
-rw-r--r-- 1 alekhya alekhya 2297 Aug 17 04:59 task4_escape.c
-rwxr-xr-x 1 alekhya alekhya 16712 Aug 7 08:21 task4_history
-rw-r--r-- 1 alekhya alekhya 3976 Aug 7 08:19 task4_history.c
-rwxr-xr-x 1 alekhya alekhya 16472 Aug 7 08:30 task5_memory
-rw-r--r-- 1 alekhya alekhya 2612 Aug 7 08:30 task5_memory.c
-rwxr-xr-x 1 alekhya alekhya 16696 Aug 17 05:03 task5_process
-rw-r--r-- 1 alekhya alekhya 2092 Aug 17 05:03 task5_process.c
drwxr-xr-x 2 alekhya alekhya 4096 Aug 1 09:08 tests
```

Child exited with status: 0

myshell\$ exit

Shell terminated.

alekhya@Alekhyareddy:~/OSSP\$

```
alekhya@Alekhyareddy: ~/OS  ×  +  ▾
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 256
#define MAX_ARGS 20

void parse_command(char *input, char *args[])
{
    int argc = 0;

    char *token = strtok(input, " ");

    while (token != NULL && argc < MAX_ARGS - 1)
    {
        args[argc] = token;
        argc++;

        token = strtok(NULL, " ");
    }

    args[argc] = NULL;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1)
    {
        printf("myshell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        /* Parse command and arguments */
        parse_command(input, args);

        if (args[0] == NULL)
            continue;
    }
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut
^X Exit      ^R Read File  ^\ Replace    ^U Paste
```

```
alekhya@Alekhyareddy: ~/OS  ×  +  ▾
GNU nano 7.2
pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
    continue;
}

/* Child process */
if (pid == 0)
{
    printf("Child Process PID: %d\n", getpid());

    /* Execute command */
    execvp(args[0], args);

    /* Runs only if execvp() fails */
    perror("Command execution failed");

    exit(2);
}

/* Parent process */
else
{
    int status;

    printf("Parent Process PID: %d\n", getpid());
    printf("Created Child PID: %d\n", pid);

    /* Wait for child */
    waitpid(pid, &status, 0);

    /* Get child exit status */
    if (WIFEXITED(status))
    {
        printf("Child exited with status: %d\n",
            WEXITSTATUS(status));
    }
    else
    {
        printf("Child terminated abnormally.\n");
    }

    printf("\n");
}

printf("Shell terminated.\n");

return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut
^X Exit      ^R Read File  ^\ Replace    ^U Paste
```